Southern New Hampshire University

MAT-350: Project 1— Matrix Theory Applied to Computer Network Engineering

Prepared on: July 22, 2026
Prepared for: Prof. K. B. Gardner
Prepared by: Alexander Ahmann

Contents

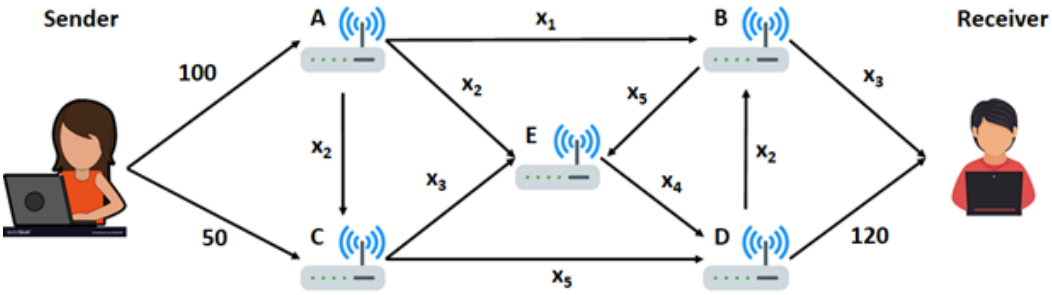

Figure 1: Topology of the Given Network

1 Introduction

Linear algebra can be seen as a generalisation of the elementary algebra typically taught in high schools,[1] and it has shown itself to be extremely useful in the sciences and technical fields (Christensen, 2012). In particular, linear algebra finds itself extremely useful in the computer sciences, such as when applied to computer graphics, cryptography, and, in the case of this paper, network engineering.

In the case of the latter subject matter, I am assuming the role of a network engineer who is tasked with determining what the data rates are, and ensuring that they do not exceed the networking hardware's capacity. A schematic of the network topology is given, and I will apply linear algebra techniques to working out what the data rate ought to be, and how it will exceed maximum capacity if data transfer goes beyond a specific threshold.

1.1 Problem Setup

Figure 1 depicts the topology of the given network to analyse. From this, I can tell that the network consists of five routers and two client nodes. The two client nodes are simply labeled "sender" and "receiver." The five routers are identified through their letters A, B, C, D, and E. The directed connections are given by x_i, where the i-subscript is a particular instance of a connection from a router to another.

The sender transmits 100 *megabits per second* (Mbps) of data to A and another 50 Mbps of data to B, and the receiver gets 120 Mbps of data from D. The data flow in, starting from the sender, is required to equal the data flow out of the network. Each router can be treated as a "junction" in the network, and each directed edge connecting the network can be interpreted

[1] At least in the American school system.

as data flowing in from a source, to data being transmitted to the reciever as a "sink."

The task at hand it to subject figure 1 into its formal treatment as a system of equations, and then apply various matrix algebraic operations against the system to work out the optimal solution for appropriate data flow.

2 Proposed Solution

2.1 Q1: Network Topology as a System of Equations

In accordance with the first directive,[2] the following is a formal treatment of figure 1 to a linear system of equations:

$$\begin{cases} \text{(A)} & x_1 + 2x_2 = 100 \\ \text{(B)} & x_1 + x_2 - x_3 - x_5 = 0 \\ \text{(C)} & x_2 - x_3 - x_5 = -50 \\ \text{(D)} & -x_2 + x_4 + x_5 = 120 \\ \text{(E)} & x_2 + x_3 - x_4 + x_5 = 0 \end{cases} \tag{1}$$

The system is formalised by treating each router as a node in a formal graph, and each ingoing connection and outgoing connection as a directed, acyclical edge from sender to reciever, and router-nodes as "junctions" in the process of transmitting information. Reorganization of the system of equations into a coefficient matrix A, a column vector of unknown variables $\vec{x}$, and a column vector of constants $\vec{b}$, and simplifying the equations and stripping the units,[3] the following is the resuling system of equations in matrix form, or written out as $A\vec{x} = \vec{b}$,[4]

[2] MAT-350 (n.d., Step 1)

[3] I strip the rate of transmission units [Mbps] for the sake of simplicity.

[4] (1) While it is implied that real numbers > 0 are positive and cases when the real number is $= 0$, (i.e. no $+$ or $\pm$-signs needed), I have nonetheless added $+$ or $\pm$-signs to "keep the terms symetrical" which will hopefully mitigate any confusions, to benefit myself and hopefully other readers. (2) Acknowledgements to Prof. K. B. Gardner for their assistance in correcting mistakes that I made when formalizing the network topology into a system of linear equations.

$$\underbrace{\begin{bmatrix} +1 & +2 & \pm 0 & \pm 0 & \pm 0 \\ +1 & +1 & -1 & \pm 0 & -1 \\ \pm 0 & +1 & -1 & \pm 0 & -1 \\ \pm 0 & -1 & \pm 0 & +1 & +1 \\ \pm 0 & +1 & +1 & -1 & +1 \end{bmatrix}}_{A} \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix}}_{\vec{x}} = \underbrace{\begin{bmatrix} +100 \\ \pm 0 \\ -50 \\ +120 \\ \pm 0 \end{bmatrix}}_{\vec{b}} \tag{2}$$

With a formal treatment of the network topology into a system of equations expressed in matrix form, I can now proceed with computational techniques with the MATLAB programming language.

2.2 Q2: The Network Topology Matrix in MATLAB

> "Use MATLAB to construct the augmented matrix $[A|\vec{b}]$. An augmented matrix with the matrix A as the first entry and the vector $\vec{b}$ as the second entry and then perform row reduction using the `rref()` function. Write out your reduced matrix and identify the free and basic variables of the system."—MAT-350 (n.d., Step 2)

The given Q2 instructs to take the formalised treatment of the network topology, express them as a matrix form in the MATLAB programming language, and then perform the basic operations of matrix augmentation and the *reduced row echelon form* procedure on the given system. The following is the solution that I presented to this problem:

```
% Solution to Question 2
A = [
    1 2 0 0 0;
    1 1 -1 0 -1;
    0 1 -1 0 -1;
    0 -1 0 1 1;
    0 1 1 -1 1
]

b = [
    100;
    0;
    -50;
    120;
```

```
    0
]

augAb = [A b]
[rrAugAb, prAugAb] = rref(augAb)
```

The `A` variable stores the A-coefficient matrix, the `b` variable stores the constants-vector $\vec{b}$ of the system, and the `augAb` variable makess the A-coefficient matrix and the $\vec{b}$ constant-vectors into an augmented matrix. Finally, the `rref(augAb)` takes the augmented matrix, and reduces it to row echelon form, which is a crucial step for solving the system of linear equations.

Running this code results in the following output:

```
A =

     1     2     0     0     0
     1     1    -1     0    -1
     0     1    -1     0    -1
     0    -1     0     1     1
     0     1     1    -1     1


b =

   100
     0
   -50
   120
     0


augAb =

     1     2     0     0     0   100
     1     1    -1     0    -1     0
     0     1    -1     0    -1   -50
     0    -1     0     1     1   120
     0     1     1    -1     1     0


rrAugAb =
```

```
     1     0     0     0     0    50
     0     1     0     0     0    25
     0     0     1     0     0    30
     0     0     0     1     0   100
     0     0     0     0     1    45


prAugAb =

     1     2     3     4     5
```

Results of the reduced row in echelon form for the augmentation of the coefficient matrix and the column-vector are stored in the `rrAugAb` variable, and said augmented matrix's pivot variables are stored in `prAugAb`. Now, from `prAugAb`, I have identified the free variables as x_1, x_2, x_3, x_4 and x_5.

2.3 Q3: LU Decomposition of the Coefficient Matrix

> "Use MATLAB to compute the LU decomposition of A, i.e., find $A = LU$. For this decomposition, find the transformed set of equations $L\mathbf{y} = \mathbf{b}$. Solve the system of equations $L\mathbf{y} = \mathbf{b}$ for the unknown vector $\mathbf{y}$."—MAT-350 (n.d., Step 3)

This step involves decomposing the coefficient matrix a product of two factors, the "lower triangular matrix" L, and the "upper triangular matrix" U. This procedure produces a pair of matricies such that:

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} l_{11} & 0 & 0 \\ l_{21} & l_{22} & 0 \\ l_{31} & l_{32} & l_{33} \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix} \tag{3}$$

Or, that

$$A = L \times U \tag{4}$$

The following is my solution to this problem:

```
% Solution to Question 3
[A_L A_U] = lu(A)
y = A_L\b
```

In particular, the `lu(A)` is a built in MATLAB function that takes in a matrix and returns its LU-decomposition. In this case, I have stored them in variables `A_L` and `A_U`. The response variable is then "solved for" by taking the lower triangular matrix and "dividing it" by the column-vector. The results of running this code can be viewed in the appendix.

2.4 Q4: Inverse of the U-matrix

> "Use MATLAB to compute the inverse of U. The matrix Uusing the `inv()` function."—MAT-350 (n.d., Step 4)

The solution to this problem is fairly simple: I just need to compute the inverse of the upper triangular matrix with the `inv(A_U)` command. The full command can be jotted down in just one line:

```
% Solution to Question 4
AU_inv = inv(A_U)
```

That is, the inverse of the upper triangulation of the coefficient matrix, A_U^{-1}, is worked out and its solution is stored in the variable `AU_inv`. This results of this computation can be viewed in the appendix, and will show itself to be a key component to solving the network topology equations for the next question.

2.5 Q5: Solution to the "Network Topology Equations"

> "Compute the solution to the original system of equations by transforming y into x i.e., compute $\mathbf{x} = U^{-1}\mathbf{y}$."—MAT-350 (n.d., Step 5)

This is the real "mean and potatoes" of this project. And the solution to this question fairly elegant: like with the previous question, the solution is a simple "one-liner" where the results of the A_U^{-1}, which are stored in `AU_inv`, and then multiplies them by the `y`-response variable vector with the A_U^{-1},

```
% Solution to Question 5
x = AU_inv * y
```

The results are in the appendix, and a rendering of the results are,

$$\vec{x} = \begin{bmatrix} 50 \\ 25 \\ 30 \\ 100 \\ 45 \end{bmatrix} \tag{5}$$

That is, when putting these values into their respective variables in the system, they will result in the constant on the other side of the equation.

3 Discussion

3.1 Q6: Verification of x_1 solution with Cramer's Rule

Cramer's rule posits that given the following relationship between the coefficient matrix A, the vector of unknowns $\vec{x}$, and the constants matrix $\vec{b}$,

$$A\vec{x} = \vec{b} \tag{6}$$

with the usual assumptions consisting of a square matrix for the coefficient matrix, and the unknown variables and column-vectors are a $1 \times n$,[5] that the solution is:

$$x_i = \frac{\det(A_k)}{\det(A)} \ (\forall k \in 1, ..., n) \tag{7}$$

where $A_{x.k.}$ is a variant of the A coefficient matrix where the k^{th} column is replace by the constant column-vector $\vec{b}$. Now, the solution has already been worked in the previous question. But, this can be a form of "double checking" of solutions to ensure reliability. The logic is that if the solution from the previous question is equal to the solutions derived from an application of Cramer's rule for each of the x_k or x_i variables, then there is good reason that the implementations are correct.

The following is a solution to solve the system specifically for x_1,

```
% Solution to Question 6
A_x1 = A
A_x1(:, 1) = b
cr_x1 = det(A_x1) / det(A)
```

The following is the results of running this code:

[5]Where n = the number of columns and rows in the coefficient matrix.

```
A_x1 =

     1     2     0     0     0
     1     1    -1     0    -1
     0     1    -1     0    -1
     0    -1     0     1     1
     0     1     1    -1     1


A_x1 =

   100     2     0     0     0
     0     1    -1     0    -1
   -50     1    -1     0    -1
   120    -1     0     1     1
     0     1     1    -1     1


cr_x1 =

   50.0000
```

The first instance of `A_x1` shows the matrix being copied from the initially set coefficient matrix `A`, then replacing the first column in the coefficient matrix `A` with the column vector `b`. Finally, the determinant of the `A_x1` is divided by the determinant of the `A`, and the solution for x_1 is worked and stored in `cr_x1`. In this case, the solution worked out from the solution from the LU-decomposition, $x_{i=1} = 50$, is equal to the solution from Cramer's rule, `cr_x1 = 50.0000`.

For the sake of "due dilegence," I have repeated the process for the other $x_{i/k}$ variables in the coefficient matrix, and the following list the results:

- For x_2: LU-decomposition solution (25) and `cr_x2 = 25.0000` are equal.
- For x_3: LU-decomposition solution (30) and `cr_x3 = 30.0000` are equal.
- For x_4: LU-decomposition solution (100) and `cr_x4 = 100` are equal.
- For x_5: LU-decomposition solution (45) and `cr_x5 = 45` are equal.

This demonstrates both the procedure for working out the solution to $x_{1..5}$ with both the "LU-decomposition" matrix algebra techniques and with Cramer's rule, and that the solutions that I have worked out are, from a software development testing perspective, reliable.

3.2 Q7: Assessment of the Network Topology Setup

With the solutions worked, and cross-confirmed, by both the "LU-decomposition" method and Cramer's rule, they need to be interpreted in the context of the original problem: to describe the "current" state of network flow, work out if they are "overloaded" or otherwise, and to make recommendations.

The following are a numbered list of recommendations that are made in table 1,

1. Do nothing.
2. Consider upgrading.
3. Upgrade connection.
4. Decrease rate of data transmission.

The number in the recommendation column in table 1 is to be mapped onto an entry in this list.

Network Link	**Recommended Capacity**	**Solution**	**Recommendation**
x_1	60	50	2
x_2	50	25	1
x_3	100	30	1
x_4	100	100	3 or 4
x_5	50	45	2

Table 1: Results Table after MAT-350 (n.d.).

In the case of network links x_1 and x_5, I recommend consider upgrading the link because their connection rates, in my judgement, approaching the recommended capacity of the network. In the case of x_4, I recommmend either upgrading the connection or decreasing the rate of data transmission, because the current data transmission rate is equal to the recommended capacity, and a small increase in the rate of data transmission can cause a breakdown. Finally, in the case of the x_2 and x_3, I recommend "doing nothing" because the data transmision rate is far below the recommend capacity, and resources ought to be focused elsewhere.

4 Conclusions

In this MATLAB project, I have demonstrated that given a network topology as depicted in figure 1, it is possible to analyse said network with the tools of matrix algebra. Indeed, linear algebra is very useful for graph theory (Johnson, 2017), and perhaps its most successful application to graph theory is shown in Google's *PageRank* algorithm (Brin & Page, 1998).

I have shown that, given the network topology depicted in figure 1,

- Given the network topology, it can be expressed in terms of a system of equations and matrix equations.
- Given the rate by which data is transmitted from the sender and reciever, it is possible to use LU-decomposition matrix algebra and Cramer's rule to solve for the unknowns in the system, and to "double check" the results for the sake of reliability.
- Network links x_2 and x_3 do not need any action done.
- Network links x_1, x_4, and x_5 need to either be upgraded, have their rate of data transmission decreased, or at the very least considered upgrading.

4.1 Acknowledgements

"David Friday" for their video footage ("David Friday", May 17-2021) demonstrating a method of formalisation of a network topology, which I have applied to initial attempts to set up the given problem, and Professor K. B. Gardner for her guidance— in particular explaining how to properly set up the system of equations.

References

Brin, S., & Page, L. (1998). The anatomy of a large-scale hypertextual Web search engine. Computer Networks and ISDN Systems, 30(1–7), 107–117. https://doi.org/10.1016/s0169-7552(98)00110-x

Christensen, J. (2012). *A Brief History of Linear Algebra*. Department of Mathematics. College of Science. The University of Utah. Retrieved on Jul. 15, 2026 from: https://www.math.utah.edu/∼gustafso/s2012/2270/web-projects/christensen-HistoryLinearAlgebra.pdf

"David Friday" (May 17, 2021). *Creating a system of equations describing flow through a network*. Retrieved on Jul. 18, 2026 from: https://www.youtube.com/watch?v=Flpwjmg8XOE

Johnson, D. (2017). *Graph theory and linear algebra*. Retrieved on Jul. 22, 2026 from: https://www.math.utah.edu/∼gustafso/s2017/2270/projects-2017/dylanJohnson/Dylan%20Johnson%20Graph%20Theory%20and%20Linear%20Algebra.pdf

MAT-350 (n.d.). *Project One Guidelines and Rubric*. Southern New Hampshire University.

A Appendix: Solution MATLAB Code

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Applied Linear Algebra (MAT-350): Project 1 Solution  %
% By Alexander Ahmann <alexander.ahmann@snhu.edu>       %
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% Q2: Use MATLAB to construct the augmented matrix [A b]
  % and then perform row reduction using the rref() function.
  % Write out your reduced matrix and identify the free and
  % basic variables of the system.

A = [
    1 2 0 0 0;
    1 1 -1 0 -1;
    0 1 -1 0 -1;
    0 -1 0 1 1;
    0 1 1 -1 1
] % The coefficient matrix

b = [
    100;
    0;
    -50;
    120;
    0
] % The column vector
```

```
augAb = [A b] % Augmented matrix of the coefficient
[rrAugAb, prAugAb] = rref(augAb) % Automatic procedure
  %  by which to reduce the [A b] matrix into an
  % echelon form.

% Q3: Use MATLAB to compute the LU  decomposition of
  % A, i.e., find A = LU. For this decomposition,
  % find the transformed set of equations Ly = b,
  % where y = Ux. Solve the system of equations
  % Ly = b for the unknown vector y.

[A_L A_U] = lu(A) % Lower-Upper decomposition
  %of coefficient matrix.
y = A_L\b % Solving for the "response variable"
  % in terms of the coefficient matrix's lower
  % triangulation matrix "divided" by the coef-
  % -ficient vector "b".

% Q4: Use MATLAB to compute the inverse of U
  % using the inv() function.

AU_inv = inv(A_U) % Automatic inverse of the coefficient
  % matrix's "upper"  triangulation matrix, or its
  % "U"-matrix.

% Q5: Compute the solution to the original system of
  % equations by transforming y into x, i.e., computing
  % x = U^-1 * y

x = AU_inv * y % Solving for the "unknowns-vector" by
  % multiplying the inverse of A_U (AU_inv) and the
  % previously discovered "y"-vector

% Q6: Check your answer for x_1 using Cramer's Rule.

A_x1 = A % create a copy of the coefficient matrix "A"
A_x1(:, 1) = b % replace the first column with the
  % constants vector "b"
cr_x1 = det(A_x1) / det(A) % Solve with Cramer's Rule
```

```
  % For good measure, check the other variables x_2,
    % x_3, ... x_5 with Cramer's rule.

A_x2 = A % create a copy of the coefficient matrix "A"
A_x2(:, 2) = b % replace the second column with the
  % constants vector "b"
cr_x2 = det(A_x2) / det(A) % Solve with Cramer's Rule

A_x3 = A % create a copy of the coefficient matrix "A"
A_x3(:, 3) = b % replace the third column with the
  % constants vector "b"
cr_x3 = det(A_x3) / det(A) % Solve with Cramer's Rule

A_x4 = A % create a copy of the coefficient matrix "A"
A_x4(:, 4) = b % replace the fourth column with the
  % constants vector "b"
cr_x4 = det(A_x4) / det(A) % Solve with Cramer's Rule

A_x5 = A % create a copy of the coefficient matrix "A"
A_x5(:, 5) = b % replace the fifth column with the
  % constants vector "b"
cr_x5 = det(A_x5) / det(A) % Solve with Cramer's Rule
```

B Appendix: Results of the MATLAB Solution

```
>> project1

A =

     1     2     0     0     0
     1     1    -1     0    -1
     0     1    -1     0    -1
     0    -1     0     1     1
     0     1     1    -1     1


b =
```

```
   100
     0
   -50
   120
     0


augAb =

     1     2     0     0     0   100
     1     1    -1     0    -1     0
     0     1    -1     0    -1   -50
     0    -1     0     1     1   120
     0     1     1    -1     1     0


rrAugAb =

     1     0     0     0     0    50
     0     1     0     0     0    25
     0     0     1     0     0    30
     0     0     0     1     0   100
     0     0     0     0     1    45


prAugAb =

     1     2     3     4     5


A_L =

    1.0000         0         0         0         0
    1.0000    1.0000         0         0         0
         0   -1.0000    1.0000         0         0
         0    1.0000   -0.5000    1.0000         0
         0   -1.0000         0   -1.0000    1.0000


A_U =
```

```
     1     2     0     0     0
     0    -1    -1     0    -1
     0     0    -2     0    -2
     0     0     0     1     1
     0     0     0     0     1
```

```
y =

   100
  -100
  -150
   145
    45
```

```
AU_inv =

    1.0000    2.0000   -1.0000         0         0
         0   -1.0000    0.5000         0         0
         0         0   -0.5000         0   -1.0000
         0         0         0    1.0000   -1.0000
         0         0         0         0    1.0000
```

```
x =

    50
    25
    30
   100
    45
```

```
A_x1 =

     1     2     0     0     0
     1     1    -1     0    -1
     0     1    -1     0    -1
     0    -1     0     1     1
```

```
     0     1     1    -1     1


A_x1 =

   100     2     0     0     0
     0     1    -1     0    -1
   -50     1    -1     0    -1
   120    -1     0     1     1
     0     1     1    -1     1


cr_x1 =

   50.0000


A_x2 =

     1     2     0     0     0
     1     1    -1     0    -1
     0     1    -1     0    -1
     0    -1     0     1     1
     0     1     1    -1     1


A_x2 =

     1   100     0     0     0
     1     0    -1     0    -1
     0   -50    -1     0    -1
     0   120     0     1     1
     0     0     1    -1     1


cr_x2 =

   25.0000


A_x3 =
```

```
     1     2     0     0     0
     1     1    -1     0    -1
     0     1    -1     0    -1
     0    -1     0     1     1
     0     1     1    -1     1


A_x3 =

     1     2   100     0     0
     1     1     0     0    -1
     0     1   -50     0    -1
     0    -1   120     1     1
     0     1     0    -1     1


cr_x3 =

   30.0000


A_x4 =

     1     2     0     0     0
     1     1    -1     0    -1
     0     1    -1     0    -1
     0    -1     0     1     1
     0     1     1    -1     1


A_x4 =

     1     2     0   100     0
     1     1    -1     0    -1
     0     1    -1   -50    -1
     0    -1     0   120     1
     0     1     1     0     1


cr_x4 =
```

```
   100


A_x5 =

     1     2     0     0     0
     1     1    -1     0    -1
     0     1    -1     0    -1
     0    -1     0     1     1
     0     1     1    -1     1


A_x5 =

     1     2     0     0   100
     1     1    -1     0     0
     0     1    -1     0   -50
     0    -1     0     1   120
     0     1     1    -1     0


cr_x5 =

    45

>>
```